

Unit: 1

Overview of OOP and Java

❖ Programming Paradigm Overview

Q. What is a Programming Paradigm?

Ans. A programming paradigm is a method or way of designing and writing computer programs.

Example: Think of going from one city to another. You can travel by:

- Car
- Train
- Airplane

➤ All these methods achieve the same goal (reaching the destination), but the approach is different. Similarly, different programming paradigms solve problems in different ways.

✓ Major Programming Paradigms

1. Procedural Programming
2. Object-Oriented Programming (OOP)
3. Functional Programming
4. Event-Driven programming

1. Procedural Programming: Procedural Programming is a programming paradigm in which a program is divided into functions (procedures) that are executed step by step to solve a problem.

Example:

```
#include <stdio.h>

void greet() {
    printf("Hello, World!");
}

int main() {
    greet();
    return 0;
}
```

Examples of Procedural Languages: C , Pascal

2. Object-Oriented Programming (OOP): Object-Oriented Programming is a programming approach in which data and functions are combined into objects to model real-world entities and provide features like encapsulation, inheritance, polymorphism, and abstraction.

Q. Why OOPS ?

Ans. As programs become larger and more complex, procedural programming becomes difficult to maintain. OOP helps in:

- Code reusability
- Better security
- Easier maintenance
- Modularity
- Reduced code duplication
- Real-world modeling

3. Functional Programming:

Functional Programming (FP) is a programming paradigm where programs are written using functions. It focuses on what to do rather than how to do it. Functions take input, process it, and return an output without changing external data.

Key Features:

- Uses pure functions
- Supports immutable data
- Uses Lambda Expressions
- Uses Functional Interfaces
- Uses Stream API (Java 8)

Example:

```
list.stream()  
    .filter(n -> n % 2 == 0)  
    .forEach(System.out::println);
```

Advantages:

- Cleaner and shorter code
- Easy to read and maintain
- Fewer bugs
- Better support for parallel programming

1. Event-Driven Programming

Event-Driven Programming is a programming paradigm in which the flow of the program is controlled by events, such as button clicks, mouse movements, keyboard input, or user actions.

Examples of Events:

- Button click
- Mouse click
- Key press
- Window close

Example:

When a user clicks the Submit button in a Java GUI application, an event is generated, and the corresponding event handler executes.

Advantages:

- Interactive and user-friendly
- Easy to build GUI applications
- Responds to user actions efficiently.

❖ Difference Between Procedural Programming and OOP

Procedural Programming

Focuses on functions/procedures.

Follows a top-down approach.

Data and functions are separate.

Less secure as data is easily accessible.

Code reusability is limited.

Suitable for small programs.

Object-Oriented Programming (OOP)

Focuses on objects and classes.

Follows a bottom-up approach.

Data and functions are combined into objects.

More secure through data encapsulation.

Supports code reusability using inheritance.

Suitable for large and complex applications.

❖ Features (Pillars) of OOP

✓ Basic Terminologies of OOPS

1. Class: A class is a blueprint or template for creating objects.

Example:

```
class Student {  
    String name;  
    int age;  
}
```

2. Object: An object is an instance of a class.

```
Student s1 = new Student();
```

➤ Here, s1 is an object of the Student class.

❖ Features (Pillars) of OOP

➤ Object-Oriented Programming (OOP) is based on four pillars:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

➤ The four pillars of OOP are Encapsulation, Inheritance, Polymorphism, and Abstraction, which together provide security, reusability, flexibility, and simplicity in software development.

❖ History of Java

- Java was developed by James Gosling and his team at Sun Microsystems in 1991.
- The project was initially called Green Project.
- Java was first named Oak, after an oak tree outside James Gosling's office.
- Since the name Oak was already trademarked, it was renamed Java (inspired by Java coffee).
- Java was officially released by Sun Microsystems in 1995.
- The main goal of Java was "Write Once, Run Anywhere (WORA)," meaning Java programs can run on any platform that has a Java Virtual Machine (JVM).

✓ Key Milestones

- 1991: Green Project started.
- 1995: Java 1.0 officially released.
- 2010: Oracle Corporation acquired Sun Microsystems and became the owner of Java.

❖ Features of Java

Java is a high-level, object-oriented, platform-independent programming language. The following are the main features of Java.

1. Simple:

Java is easy to learn because it has a simple syntax and removes complex features like pointers and operator overloading.

Example: Learning Java is like driving an automatic car—it is easier than driving a manual car.

2. Object-Oriented:

Java organizes programs using classes and objects, making code reusable and easy to maintain.

Example: A Car is an object with:

- Properties: Color, Model, Speed
- Methods: Start(), Stop(), Brake()

3. Platform Independent:

Java programs can run on any operating system because Java code is converted into bytecode, which is executed by the JVM.

Example: A PDF file can be opened on Windows, Linux, or macOS without changing the file.

Java Principle: Write Once, Run Anywhere (WORA).

4. Portable:

Java programs can be moved from one system to another without modification.

Example: A USB pen drive works on different computers without changing the files.

5. Secure:

Java provides built-in security features such as bytecode verification, class loader, and no direct memory access.

Example: An ATM allows only authorized users to access their accounts after entering the correct PIN.

6. Robust:

Java is reliable because it provides exception handling, automatic memory management (Garbage Collection), and strong type checking.

Example: A car with airbags protects passengers even if an accident occurs.

7. Multithreaded:

Java allows multiple tasks to run at the same time using threads.

Example: While using your smartphone, you can:

- Listen to music
- Download a file
- Chat on WhatsApp

➤ All these tasks run simultaneously.

8. Distributed:

Java supports applications that work over a network.

Example: Online banking connects your mobile app to the bank's server through the Internet.

9. High Performance:

Java uses the Just-In-Time (JIT) Compiler to improve execution speed.

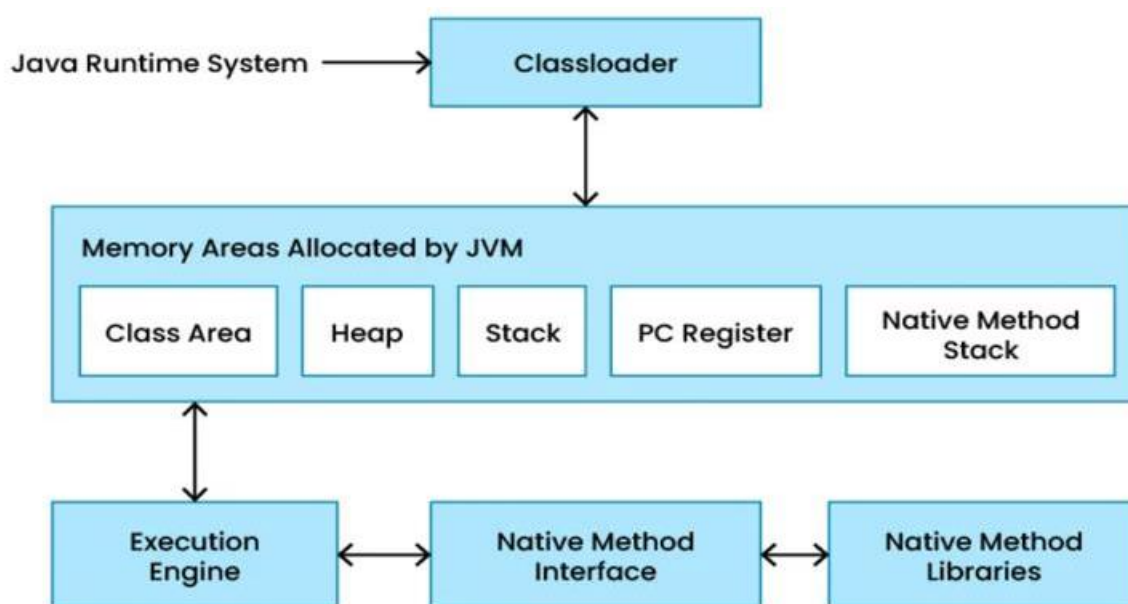
Example: A GPS navigation app responds faster after learning frequently used routes.

10. Dynamic:

Java can load classes and libraries during program execution.

Example: A mobile app downloads new features through an update without reinstalling the entire application.

❖ JVM Architecture



❖ JVM, JRE and JDK

➤ JVM (Java Virtual Machine)

- JVM is a virtual machine that runs Java bytecode.
- It is the engine that executes Java programs.
- JVM converts bytecode into machine code.
- It makes Java platform independent.
- JVM helps in running Java programs on different operating systems.

➤ **JRE (Java Runtime Environment)**

- JRE provides the environment required to run Java programs.
JRE = JVM + Libraries
- It is used to run Java programs.
- It is not used to write or develop code.

Example:

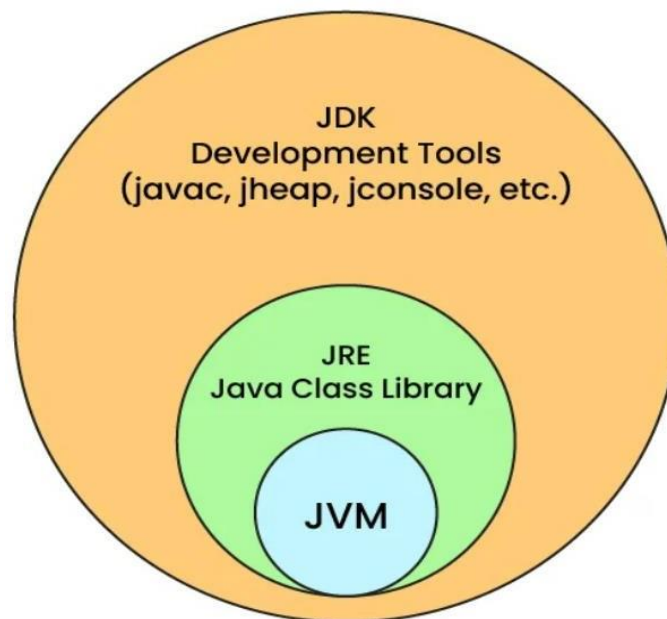
Just like a TV needs electricity to run, Java programs need JRE to run.

➤ **JDK (Java Development Kit)**

- JDK provides tools to develop Java applications.
JDK = JRE + Development Tools
- It is used by programmers to write, compile, and run Java programs.

Example:

Just like a mechanic needs tools to repair a machine, a programmer needs JDK to write code.



❖ **Java Program Structure**

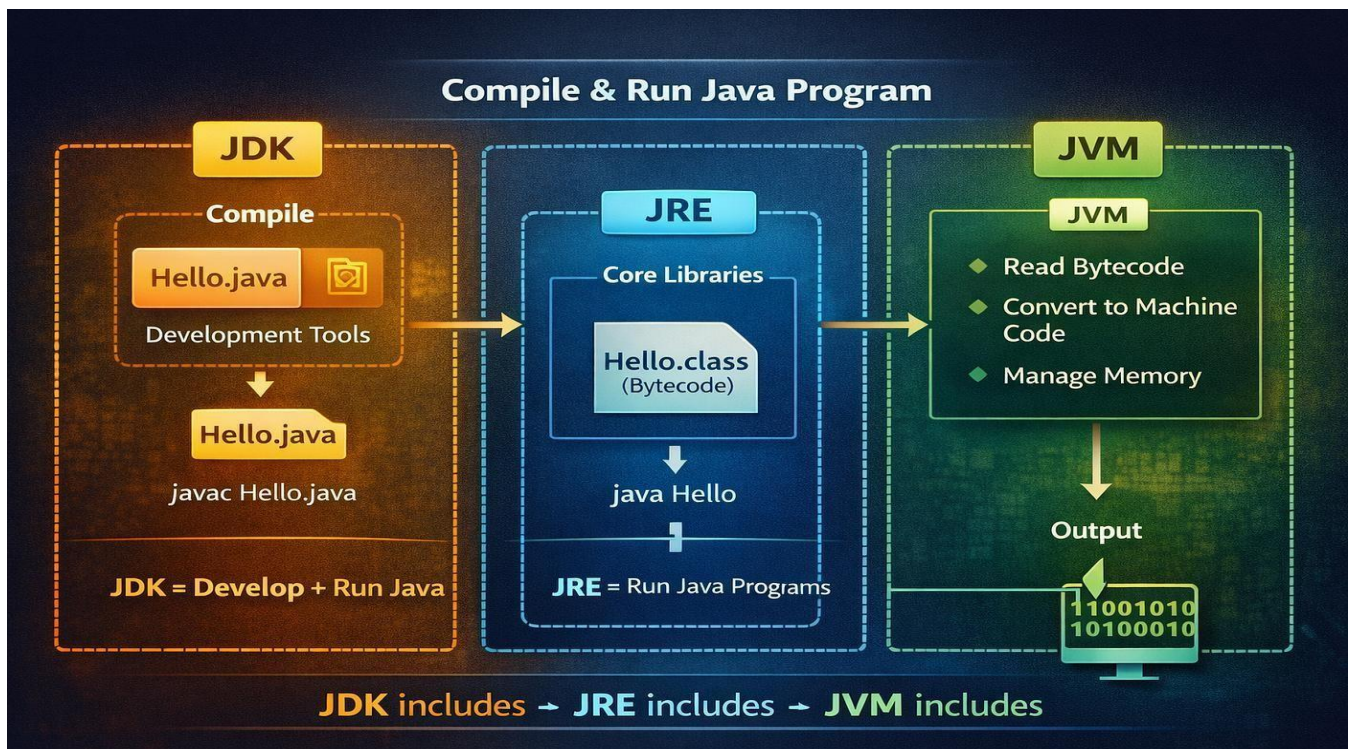
```
import java.lang.*;

Class First {
    public static void main(String[] args) {
        System.out.println("Hello, World...");
    }
}
```

- **Java.lang.*;** : It is a package
- **main()** : method to execute the program
- **static:** if you want to use class without creating an object we need to use static.
- **System.out.println();** : It is a method to print any thing.
- **String[] args:** Command line arguments used to take input from command.

Note: - File name and class name should be same.

❖ Java Compilation & Execution Process



❖ Steps Involved

1. Write the Source Code

Create a Java program and save it with a .java extension.

Example:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

➤ **File name:** Hello.java

2. Compilation

The Java Compiler (javac) converts the source code into bytecode.

```
javac Hello.java
```

Output: Hello.class

- Source Code → .java
- Compiler (javac) → Bytecode (.class)

3. Class Loader:

When the program is executed, the Class Loader loads the .class file into memory.

4. Bytecode Verification:

The Bytecode Verifier checks the bytecode for security and correctness.

5. JVM Execution:

The Java Virtual Machine (JVM) executes the bytecode.

```
java Hello
```

The JVM uses:

- ❖ **Interpreter** – Executes bytecode line by line.
- ❖ **JIT (Just-In-Time) Compiler** – Converts frequently used bytecode into machine code for faster execution.

6. Output Generation

```
Hello World
```

❖ JDK Installation steps

Step 1: Download JDK

- Visit the official [Oracle JDK Downloads](#) page.
- Select the appropriate version for your operating system (Windows, Linux, or macOS).
- Download the installer.

Step 2: Install JDK

- Double-click the downloaded installer.
- Click Next.
- Choose the installation directory (default location is usually recommended).
- Click Install.
- Wait for the installation to complete and click Close.

Step 3: Set Environment Variables (Windows)

- Add JAVA_HOME
- Open Control Panel → System → Advanced System Settings.
- Click Environment Variables.
- Under System Variables, click New.

Enter:

Variable Name: JAVA_HOME

Variable Value: Path of JDK installation, e.g.,

C:\Program Files\Java\jdk-24

Update PATH Variable

- Select Path under System Variables and click Edit.
- Click New and add: %JAVA_HOME%\bin
- Click OK to save.

Step 4: Verify Installation

- Open Command Prompt and type:
`java -version`

❖ Introduction to IDEs

Q. What is an IDE?

Ans. IDE (Integrated Development Environment) is a software application that provides all the tools required to write, compile, run, test, and debug Java programs in one place.

Instead of using separate tools for coding, compiling, and debugging, an IDE combines everything into a single interface, making Java development easier and faster.

✓ Common IDEs for Java

1. IntelliJ IDEA
2. Eclipse IDE
3. Apache NetBeans
4. Visual Studio Code (with Java Extension Pack)

✓ Main Features of an IDE

- **Code Editor** – Write and edit Java programs.
- **Compiler Integration** – Compile Java code.
- **Debugger** – Find and fix errors.
- **Syntax Highlighting** – Displays keywords, variables, and comments in different colors.

- **Auto-Completion** – Suggests code while typing.
- **Error Detection** – Highlights syntax and logical errors.
- **Project Management** – Organizes files and folders.
- **Build and Run Tools** – Compile and execute programs with a single click.

✓ **Advantages of Using an IDE**

- Makes coding faster and easier.
- Reduces programming errors.
- Improves code readability.
- Simplifies debugging.
- Increases programmer productivity.
- Helps manage large Java projects efficiently.

❖ **Important Questions**

1. What is Java?
2. Why is Java called a platform-independent programming language?
3. What is bytecode?
4. Differentiate between JVM, JRE, and JDK.
5. What is the role of the JIT (Just-In-Time) Compiler?
6. Explain the features of Java with suitable real-life examples.
7. Differentiate between Procedural Programming and Object-Oriented Programming.
8. Explain the four main features (pillars) of Object-Oriented Programming with examples.
9. Draw and explain the JVM Architecture.
10. Differentiate between JVM, JRE, and JDK with a neat diagram.
11. Explain the Java Compilation and Execution Process with a neat diagram.
12. Explain the structure of a Java program with an example.
13. Explain the history of Java and its evolution.
14. Explain the installation of JDK and the purpose of PATH and CLASSPATH variables.
15. What is an IDE? Explain the features and advantages of IDEs used for Java programming.